



Design Patterns in .NET

Short & Practical Notes

⚡ Quick Revision Before Interview · Last-Minute Refresh

Covers: Singleton · Factory · Builder · Adapter · Decorator · Facade · Strategy · Observer · Mediator · CQRS · Repository · DI · Unit of Work

With C# Examples · Diagrams · Interview Q&A · Cheat Sheet
By [Logic Lense](#)

Table of Contents

	Table of Contents.....	2
1	Introduction to Design Patterns	4
	What Are Design Patterns?	4
	Why Do Developers Use Them?	4
	Real-World Analogy	4
	Patterns in Modern .NET	4
2	Categories of Design Patterns	5
3	Singleton Pattern	6
	What Problem It Solves	6
	C# Implementation	6
	In ASP.NET Core — Singleton DI	6
4	Factory Pattern	8
	What Problem It Solves	8
	C# Implementation — Payment Gateway	8
5	Builder Pattern	10
	C# Implementation — Email Builder	10
6	Adapter Pattern	12
	C# Implementation — Payment Gateway Adapter	12
7	Decorator Pattern	14
	C# Implementation — Logging & Caching Decorator	14
8	Facade Pattern	16
	What Problem It Solves	16
	C# Implementation — Order Facade	16
9	Strategy Pattern	18
	C# Implementation — Discount Strategy	18
10	Observer Pattern	20
	C# Implementation — Order Notification	20
1 1	Mediator Pattern	22
	C# + MediatR Implementation (ASP.NET Core)	22
1 2	Common Patterns in .NET Projects	24
	Dependency Injection (DI)	24
	Repository Pattern	24
	Unit of Work Pattern	24
	CQRS Pattern (Command Query Responsibility Segregation)	25
1 3	When NOT to Use Design Patterns	26

1 4	Real Project Example — ASP.NET Core E-Commerce API	27
	Folder Structure	27
	Patterns Used in This Project	27
1 5	Interview Preparation	28
	One-Line Explanations	28
	Common Interview Questions	28
★	Quick Cheat Sheet — Revision Page.....	29

Logic Lense

1 Introduction to Design Patterns

⚡ **Purpose:** Understand what design patterns are and why they matter in modern .NET development.

What Are Design Patterns?

Design patterns are reusable solutions to commonly occurring problems in software design. They are NOT code — they are templates or blueprints that guide how to structure your code to solve recurring challenges elegantly.

Why Do Developers Use Them?

Benefit	Explanation
Common Vocabulary	Say 'Singleton' and every dev instantly understands the concept
Proven Solutions	Battle-tested approaches — no need to reinvent the wheel
Maintainability	Code structured around patterns is easier to maintain and extend
Testability	Patterns like Strategy and Repository make unit testing much simpler
Team Collaboration	Clear, shared patterns reduce miscommunication in teams
SOLID Principles	Patterns enforce Single Responsibility, Open/Closed, and other principles

Real-World Analogy

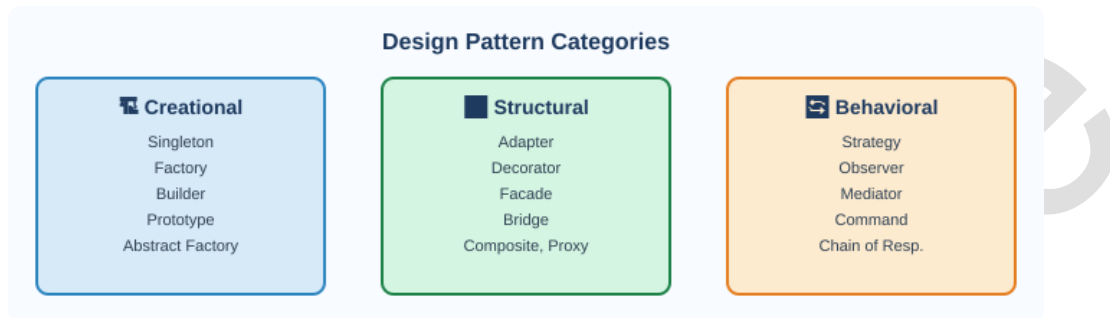
🌐 **Real-Life Analogy:** Think of a recipe book. You don't invent a new way to bake bread every time — you follow a proven recipe. Design patterns are the 'recipe book' for software architecture.

Patterns in Modern .NET

ASP.NET Core is built ON design patterns. Dependency Injection (built-in), Middleware (Chain of Responsibility), Repository Pattern, CQRS with MediatR — patterns are everywhere in the .NET ecosystem.

2 Categories of Design Patterns

⚡ **Purpose:** Know the three categories so you can identify which pattern family applies to your problem.



Category	What It Does	Key Patterns	When to Use
 Creational	Object creation mechanisms	Singleton, Factory, Builder	When object creation is complex or needs to be controlled
 Structural	Compose objects/classes	Adapter, Decorator, Facade	When you need to change how classes/objects relate
 Behavioral	Communication between objects	Strategy, Observer, Mediator	When you need to change how objects interact or behave

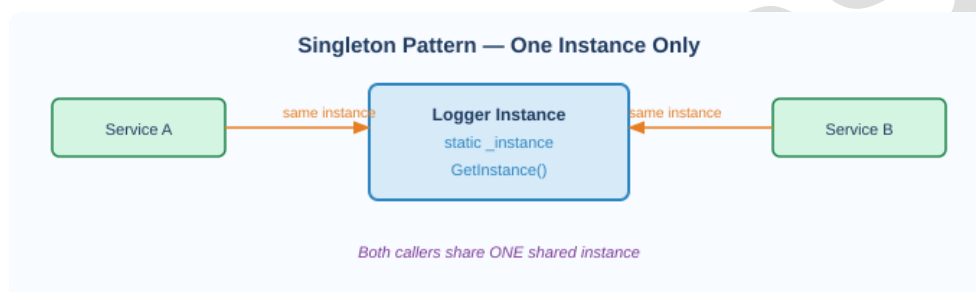
CREATIONAL DESIGN PATTERNS

3 Singleton Pattern

⚡ **Purpose:** Ensure a class has only one instance and provide a global access point to it.

What Problem It Solves

When you need exactly one shared resource — like a Logger, Configuration manager, or Database connection pool — the Singleton pattern prevents multiple instances from being accidentally created, saving memory and preventing state inconsistencies.



C# Implementation

```
// Thread-safe Singleton using Lazy<T>
public sealed class Logger
{
    private static readonly Lazy<Logger> _instance =
        new Lazy<Logger>(() => new Logger());

    // Private constructor prevents external instantiation
    private Logger() { }

    public static Logger Instance => _instance.Value;

    public void Log(string message)
        => Console.WriteLine($"[LOG] {DateTime.Now}: {message}");
}

// Usage — same instance everywhere
Logger.Instance.Log("App started");
Logger.Instance.Log("User logged in");
```

In ASP.NET Core — Singleton DI

```
// Register as Singleton in DI container
builder.Services.AddSingleton<IAppConfig, AppConfig>();

// ASP.NET Core creates ONE instance for the entire app lifetime
```

🌐 **Real-Life Analogy:** A country has exactly ONE President. No matter who asks, there is only one President instance — the same person is returned every time.

🎯 **Interview Quick Answer**

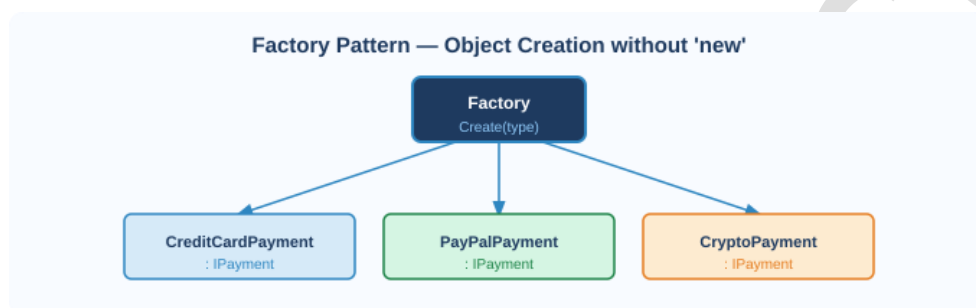
- Singleton ensures one instance exists for the lifetime of the application.
- Use sealed class + private constructor + static Lazy<T> for thread-safety.
- In .NET DI: AddSingleton() — same instance for entire app lifetime.
- Watch out: Singleton holding scoped services causes 'captive dependency' bugs.

4 Factory Pattern

⚡ **Purpose:** Create objects without exposing the creation logic; work with an interface, not a concrete class.

What Problem It Solves

When your code needs to create objects but shouldn't know exactly which class to instantiate. The factory decides what to create based on input — your code only knows the interface.



C# Implementation — Payment Gateway

```

public interface IPaymentMethod
{
    void ProcessPayment(decimal amount);
}

public class CreditCardPayment : IPaymentMethod
{
    public void ProcessPayment(decimal amount)
        => Console.WriteLine($"Charged ${amount} via Credit Card");
}

public class PayPalPayment : IPaymentMethod
{
    public void ProcessPayment(decimal amount)
        => Console.WriteLine($"Sent ${amount} via PayPal");
}

// Factory
public static class PaymentFactory
{
    public static IPaymentMethod Create(string type) => type switch
    {
        "creditcard" => new CreditCardPayment(),
        "paypal"     => new PayPalPayment(),
        _            => throw new NotSupportedException($"Unknown: {type}")
    };
}

// Usage - caller doesn't care about the concrete class
var payment = PaymentFactory.Create("paypal");
payment.ProcessPayment(99.99m);
  
```

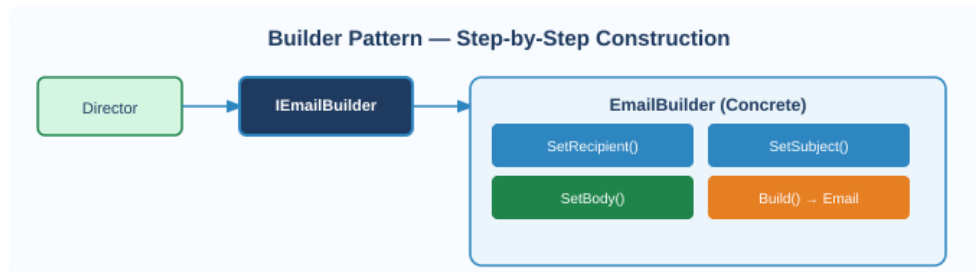
 **Real-Life Analogy:** A car rental company lets you book 'Sedan' or 'SUV'. You don't build the car — the rental company (factory) gives you the right type.

 Interview Quick Answer

- Factory encapsulates object creation — caller uses `IPayment`, not `CreditCardPayment`.
- Follows Open/Closed: add new types by adding new classes, not editing the factory.
- Difference from Abstract Factory: Abstract Factory creates families of related objects.
- Common in .NET: `IHttpClientFactory`, `ILoggerFactory`.

5 Builder Pattern

⚡ **Purpose:** Construct complex objects step-by-step using a fluent/chaining API.



C# Implementation — Email Builder

```

public class Email
{
    public string To { get; set; }
    public string Subject { get; set; }
    public string Body { get; set; }
    public bool IsHtml { get; set; }
}

public class EmailBuilder
{
    private readonly Email _email = new();

    public EmailBuilder To(string email) { _email.To = email; return this; }
    public EmailBuilder Subject(string subject) { _email.Subject = subject; return this; }
    public EmailBuilder Body(string body) { _email.Body = body; return this; }
    public EmailBuilder AsHtml() { _email.IsHtml = true; return this; }
    public Email Build() => _email;
}

// Usage - clear, readable, step-by-step
var email = new EmailBuilder()
    .To("john@example.com")
    .Subject("Welcome!")
    .Body("<h1>Hello</h1>")
    .AsHtml()
    .Build();
    
```

🌐 **Real-Life Analogy:** Building a custom PC: choose CPU, RAM, GPU, SSD step by step. You're building a complex object (PC) by setting each optional part.

🔗 **Interview Quick Answer**

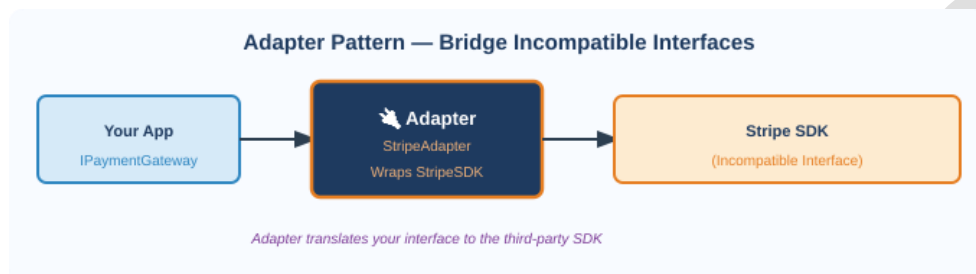
- Builder separates construction from representation — same process, different representations.
- Great for objects with many optional parameters (avoids 'telescoping constructors').
- In .NET: `WebApplication.CreateBuilder()`, `IHostBuilder`, `Stringbuilder` all use this pattern.
- Fluent builders return 'this' from each method for chaining.

Logic Lense

■ STRUCTURAL DESIGN PATTERNS

6 Adapter Pattern

⚡ **Purpose:** Make incompatible interfaces work together by wrapping one in an adapter class.



C# Implementation — Payment Gateway Adapter

```

// Your application's interface
public interface IPaymentGateway
{
    bool Charge(string customerId, decimal amount);
}

// Third-party Stripe SDK (incompatible interface)
public class StripeSDK
{
    public StripeResult ProcessCharge(string token, int amountInCents)
        => new StripeResult { Success = true };
}

// Adapter - wraps StripeSDK to match IPaymentGateway
public class StripeAdapter : IPaymentGateway
{
    private readonly StripeSDK _stripe = new();

    public bool Charge(string customerId, decimal amount)
    {
        var result = _stripe.ProcessCharge(customerId, (int)(amount * 100));
        return result.Success;
    }
}

// Usage - app only knows IPaymentGateway
IPaymentGateway gateway = new StripeAdapter();
gateway.Charge("cust_123", 49.99m);
  
```

🌐 **Real-Life Analogy:** A power adapter lets you plug a US device into a UK socket. The adapter converts the interface without changing either device.

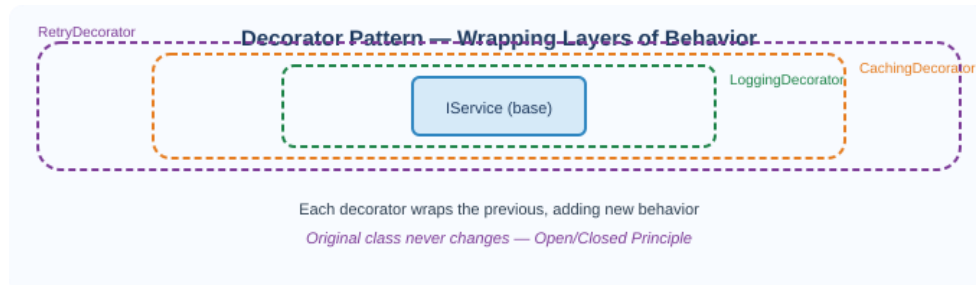
🎯 Interview Quick Answer

- Adapter wraps an incompatible class to match an expected interface.
- Also called 'Wrapper' pattern.
- Used when integrating third-party SDKs into your app's abstraction layer.
- Difference from Facade: Adapter converts interfaces; Facade simplifies them.

Logic Lense

7 Decorator Pattern

⚡ **Purpose:** Add new behavior to an object without modifying its original class — wrap it!



C# Implementation — Logging & Caching Decorator

```
public interface IProductService
{
    Product GetProduct(int id);
}

// Base implementation
public class ProductService : IProductService
{
    public Product GetProduct(int id) => _repo.Find(id);
}

// Logging Decorator — wraps IProductService
public class LoggingProductService : IProductService
{
    private readonly IProductService _inner;
    private readonly ILogger _logger;

    public LoggingProductService(IProductService inner, ILogger logger)
    {
        _inner = inner; _logger = logger;
    }

    public Product GetProduct(int id)
    {
        _logger.LogInformation("Getting product {id}", id);
        return _inner.GetProduct(id); // delegates to base
    }
}

// Register with DI — wraps the base service
services.AddScoped<IProductService>(sp =>
    new LoggingProductService(
        new ProductService(),
        sp.GetRequiredService<ILogger<LoggingProductService>>()));
```

 **Real-Life Analogy:** Adding toppings to a pizza. Base pizza exists. You wrap it with extra cheese, then with pepperoni — each topping is a decorator adding behavior without changing the original pizza.

🔗 Interview Quick Answer

- Decorator adds responsibility to objects dynamically at runtime.
- Both decorator and component implement the same interface.
- Respects Open/Closed Principle — extend without modification.
- In .NET: ASP.NET Core Middleware IS the Decorator pattern.

Logic Lense

8 Facade Pattern

⚡ **Purpose:** *Provide a simplified interface to a complex subsystem.*

What Problem It Solves

When your checkout flow requires calling 5 different services (inventory, payment, shipping, email, audit), instead of forcing the controller to orchestrate all of them, a Facade bundles them behind a single, clean method.

C# Implementation — Order Facade

```
// Complex subsystem classes
public class InventoryService { public void Reserve(int productId) { ... } }
public class PaymentService { public bool Charge(decimal amount) { ... } }
public class ShippingService { public void Schedule(int orderId) { ... } }
public class NotificationService { public void SendConfirmation(int orderId) { ... } }

// Facade - single entry point
public class OrderFacade
{
    private readonly InventoryService _inventory;
    private readonly PaymentService _payment;
    private readonly ShippingService _shipping;
    private readonly NotificationService _notify;

    public OrderFacade( /* inject all */ ) { ... }

    public bool PlaceOrder(Order order)
    {
        _inventory.Reserve(order.ProductId);
        if (!_payment.Charge(order.Total)) return false;
        _shipping.Schedule(order.Id);
        _notify.SendConfirmation(order.Id);
        return true;
    }
}

// Controller - uses ONLY the facade
[HttpPost] public IActionResult PlaceOrder(OrderDto dto)
{
    _orderFacade.PlaceOrder(dto);
    return Ok();
}
```

🌐 **Real-Life Analogy:** A restaurant waiter is your Facade. You say 'I want the steak' — the waiter coordinates kitchen, bar, and billing. You don't interact with each subsystem directly.

🎯 Interview Quick Answer

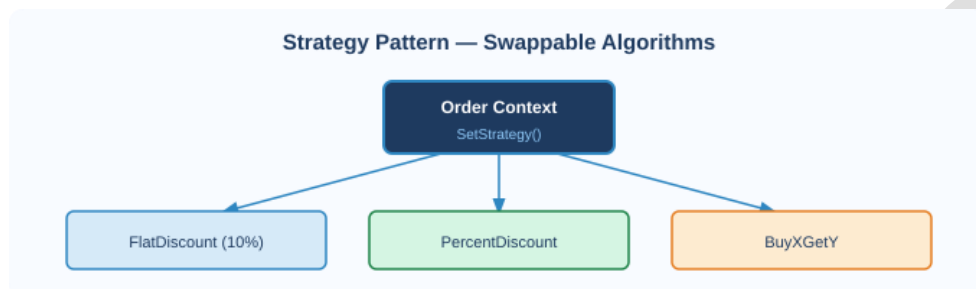
- Facade provides a simple interface over a complex set of subsystems.
- Reduces coupling between clients and the subsystem internals.
- Difference from Adapter: Facade simplifies; Adapter translates incompatible interfaces.
- Common in service layer classes that orchestrate multiple repositories/services.

Logic Lense

📦 BEHAVIORAL DESIGN PATTERNS

🔍 Strategy Pattern

⚡ **Purpose:** Define a family of algorithms, encapsulate each one, and make them interchangeable at runtime.



C# Implementation — Discount Strategy

```

// Strategy interface
public interface IDiscountStrategy
{
    decimal ApplyDiscount(decimal originalPrice);
}

// Concrete strategies
public class NoDiscount : IDiscountStrategy
{
    public decimal ApplyDiscount(decimal p) => p;
}

public class PercentDiscount : IDiscountStrategy
{
    private readonly decimal _percent;
    public PercentDiscount(decimal percent) => _percent = percent;
    public decimal ApplyDiscount(decimal p) => p * (1 - _percent / 100);
}

public class BuyTwoGetOneFree : IDiscountStrategy
{
    public decimal ApplyDiscount(decimal p) => p * 0.67m;
}

// Context — uses whichever strategy is injected
public class ShoppingCart
{
    private IDiscountStrategy _strategy = new NoDiscount();

    public void SetStrategy(IDiscountStrategy strategy)
        => _strategy = strategy;

    public decimal Checkout(decimal total)
        => _strategy.ApplyDiscount(total);
}

// Usage — swap strategies at runtime
var cart = new ShoppingCart();
cart.SetStrategy(new PercentDiscount(20));
    
```

```
Console.WriteLine(cart.Checkout(100)); // 80
```

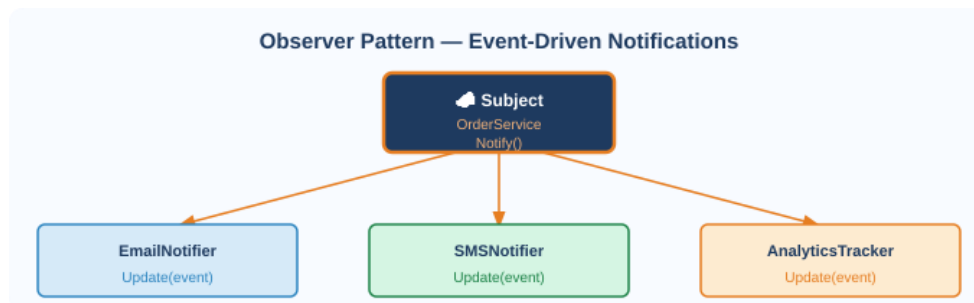
 **Real-Life Analogy:** Google Maps offers different route strategies: fastest, shortest, avoid tolls. You pick the strategy; the map uses it without changing the overall navigation code.

Interview Quick Answer

- Strategy lets you swap the algorithm a class uses without changing the class.
- Eliminates large if/switch blocks for choosing behavior.
- Follows Open/Closed: add new strategies without modifying the context.
- In .NET: `IComparer<T>`, `IEqualityComparer<T>`, sorting strategies are all Strategy pattern.

10 Observer Pattern

⚡ **Purpose:** Define a one-to-many dependency so that when one object changes state, all its dependents are notified automatically.



C# Implementation — Order Notification

```

// Observer interface
public interface IOrderObserver
{
    void OnOrderPlaced(Order order);
}

// Concrete observers
public class EmailNotifier : IOrderObserver
{
    public void OnOrderPlaced(Order order)
        => Console.WriteLine($"Email sent for order #{order.Id}");
}

public class InventoryUpdater : IOrderObserver
{
    public void OnOrderPlaced(Order order)
        => Console.WriteLine($"Stock decremented for #{order.Id}");
}

// Subject (publisher)
public class OrderService
{
    private readonly List<IOrderObserver> _observers = new();

    public void Subscribe(IOrderObserver obs) => _observers.Add(obs);
    public void Unsubscribe(IOrderObserver obs) => _observers.Remove(obs);

    public void PlaceOrder(Order order)
    {
        // ... save order ...
        foreach (var obs in _observers) obs.OnOrderPlaced(order);
    }
}

// Usage
var svc = new OrderService();
svc.Subscribe(new EmailNotifier());
svc.Subscribe(new InventoryUpdater());
svc.PlaceOrder(new Order { Id = 42 });
    
```

```
// .NET built-in: use C# events / IObservable<T>
```

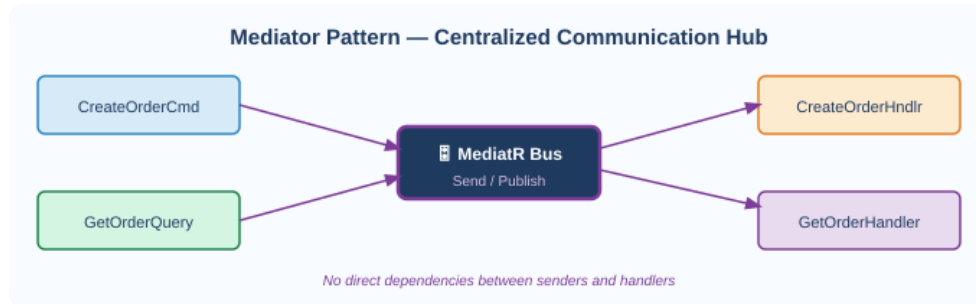
 **Real-Life Analogy:** YouTube subscriptions: When a creator uploads a video (subject), all subscribers (observers) get notified automatically. Unsubscribing removes you from the list.

Interview Quick Answer

- Observer defines a publish/subscribe relationship between objects.
- Subject maintains a list of observers and notifies all on state change.
- In .NET: C# events/delegates, IObservable<T>/IObserver<T>, EventHandler are Observer pattern.
- MediatR's INotification/INotificationHandler also implements this pattern.

1 1 Mediator Pattern

⚡ **Purpose:** Reduce chaotic dependencies between objects by having them communicate through a central mediator.



C# + MediatR Implementation (ASP.NET Core)

```
// Install: dotnet add package MediatR

// 1. Define a Command (Request)
public record CreateOrderCommand(string Product, int Qty) : IRequest<int>;

// 2. Define a Handler
public class CreateOrderHandler : IRequestHandler<CreateOrderCommand, int>
{
    private readonly IOrderRepo _repo;
    public CreateOrderHandler(IOrderRepo repo) => _repo = repo;

    public async Task<int> Handle(CreateOrderCommand cmd, CancellationToken ct)
    {
        var order = new Order { Product = cmd.Product, Qty = cmd.Qty };
        await _repo.SaveAsync(order);
        return order.Id;
    }
}

// 3. Register MediatR in DI
builder.Services.AddMediatR(cfg =>
    cfg.RegisterServicesFromAssembly(Assembly.GetExecutingAssembly()));

// 4. Use in Controller — no direct service dependency!
[HttpPost]
public async Task<IActionResult> CreateOrder(CreateOrderCommand cmd)
{
    var id = await _mediator.Send(cmd);
    return Ok(id);
}
```

🌐 **Real-Life Analogy:** Air Traffic Control tower is the mediator. Planes (objects) don't communicate with each other directly — they all communicate through ATC, which coordinates everything centrally.

🎯 Interview Quick Answer

- Mediator reduces many-to-many dependencies to one-to-many through a central hub.
- MediatR is the de-facto Mediator library in ASP.NET Core.
- Used heavily in CQRS — commands and queries go through the mediator to handlers.
- Promotes loose coupling: controllers don't depend directly on service classes.

Logic Lense

🌀 DESIGN PATTERNS IN ASP.NET CORE

1 2 Common Patterns in .NET Projects

⚡ **Purpose:** Learn the four architectural patterns every senior .NET developer must know.

Dependency Injection (DI)

Built into ASP.NET Core. Register services in Program.cs and inject via constructor. Supports Singleton, Scoped, Transient lifetimes.

```
builder.Services.AddSingleton<IConfig, AppConfig>(); // 1 instance forever
builder.Services.AddScoped<IOrderService, OrderService>(); // 1 per HTTP request
builder.Services.AddTransient<IEmailSender, SmtpSender>(); // new each time
```

Repository Pattern

Abstracts data access behind an interface. Business logic depends on IRepository<T>, not directly on DbContext. Makes unit testing easy with mock repositories.

```
public interface IOrderRepository
{
    Task<Order> GetByIdAsync(int id);
    Task<IEnumerable<Order>> GetAllAsync();
    Task AddAsync(Order order);
    Task SaveChangesAsync();
}

public class OrderRepository : IOrderRepository
{
    private readonly AppDbContext _ctx;
    public OrderRepository(AppDbContext ctx) => _ctx = ctx;
    public Task<Order> GetByIdAsync(int id) => _ctx.Orders.FindAsync(id).AsTask();
    // ... implement other methods
}
```

Unit of Work Pattern

Coordinates multiple repositories in a single transaction. Calls SaveChanges() once for all changes, ensuring atomic operations.

```
public interface IUnitOfWork : IDisposable
{
    IOrderRepository Orders { get; }
    IProductRepository Products { get; }
    Task<int> CommitAsync();
}

// Usage - all changes saved in one transaction
await _uow.Orders.AddAsync(order);
await _uow.Products.UpdateStockAsync(productId, -1);
await _uow.CommitAsync(); // single SaveChanges()
```

CQRS Pattern (Command Query Responsibility Segregation)

Separate read (Query) operations from write (Command) operations. Commands change state; Queries return data. Often implemented with MediatR in ASP.NET Core.

```
// Command – changes state, returns minimal data
public record CreateProductCommand(string Name, decimal Price) : IRequest<int>;

// Query – reads only, never changes state
public record GetProductQuery(int Id) : IRequest<ProductDto>;

// Benefits: separate read/write models, optimized DB queries,
// easy to add cross-cutting concerns (logging, validation, caching)
```

Pattern	Problem Solved	Key Benefit	.NET Implementation
DI	Tight coupling between classes	Easy to swap/test/mock	Built-in IServiceCollection
Repository	DB logic scattered in services	Clean separation of concerns	Interface + EF Core implementation
Unit of Work	Inconsistent multi-table saves	Atomic transactions	Single SaveChanges() wrapper
CQRS	Fat services doing read+write	Scalable, clear intent	MediatR IRequest/IRequestHandler

1 3 When NOT to Use Design Patterns

⚡ **Purpose:** Patterns are tools, not rules. Using them blindly leads to over-engineered, hard-to-read code.

⚠ Signs You're Over-Engineering

- You're adding patterns 'just in case' — YAGNI (You Ain't Gonna Need It).
- A 3-line function becomes 5 files and 3 interfaces.
- New devs can't understand the flow without a map.
- You have an interface with only ONE implementation, no testing benefit.
- You're implementing CQRS for a project with 5 endpoints.

✗ Pattern Misuse	☑ Better Approach
Repository over simple EF Core queries	Use DbContext directly for simple CRUD apps
Singleton for everything — global state abuse	Use Scoped/Transient where appropriate
Factory when you always create ONE known type	Use 'new' directly — factories add no value
Observer for simple sequential method calls	Just call the method directly
CQRS for a 2-table CRUD app	Use a simple service class

💡 **Golden Rule:** Start simple. Add patterns when you feel the pain of NOT having them — not before.

1 4 Real Project Example — ASP.NET Core E-Commerce API

⚡ **Purpose:** See how multiple patterns work together in a real-world project.

Folder Structure

```
src/
├── API/                                <- ASP.NET Core controllers
│   └── Controllers/
│       └── OrdersController.cs        (uses MediatR - Mediator pattern)
├── Application/                        <- Use cases / business logic
│   ├── Commands/
│   │   └── CreateOrderCommand.cs      (CQRS Command)
│   ├── Queries/
│   │   └── GetOrderQuery.cs           (CQRS Query)
│   └── Handlers/
│       ├── CreateOrderHandler.cs      (MediatR Handler)
│       └── GetOrderHandler.cs
├── Domain/                            <- Entities and interfaces
│   ├── Entities/Order.cs
│   └── Interfaces/
│       ├── IOrderRepository.cs        (Repository interface)
│       └── IUnitOfWork.cs             (Unit of Work interface)
├── Infrastructure/                    <- Concrete implementations
│   ├── Repositories/
│   │   └── OrderRepository.cs          (EF Core implementation)
│   └── Services/
│       ├── StripeAdapter.cs           (Adapter - wraps Stripe SDK)
│       └── EmailService.cs             (Singleton)
└── Shared/
    └── DTOs/                          (Data Transfer Objects)
```

Patterns Used in This Project

Pattern	Where Used	Why
Mediator (MediatR)	Controllers → Handlers	Decouple controller from business logic
CQRS	Commands vs Queries	Separate read/write concerns
Repository	OrderRepository	Decouple data access from business logic
Unit of Work	Per-request transaction	Atomic multi-table operations
Adapter	StripeAdapter	Integrate third-party payment SDK
Singleton	EmailService	Share one email client across requests
Dependency Injection	Everywhere (built-in)	Loose coupling, easy testing
Decorator	LoggingMiddleware	Add logging without changing services

1 5 Interview Preparation

⚡ **Purpose:** One-line explanations and common interview questions for each pattern.

One-Line Explanations

Pattern	One-Line Answer
Singleton	Ensures a class has exactly one instance, accessible globally.
Factory	Creates objects through a method without specifying the exact class.
Builder	Constructs complex objects step-by-step using a fluent API.
Adapter	Wraps an incompatible interface to match the one your code expects.
Decorator	Adds behavior to objects at runtime by wrapping them.
Facade	Provides a simple interface to a complex subsystem.
Strategy	Defines a family of algorithms that can be swapped at runtime.
Observer	Notifies all dependent objects automatically when a subject changes.
Mediator	Routes communication between objects through a central hub.

Common Interview Questions

Question	Key Points to Cover
What's the diff: Singleton vs Static class?	Singleton: can implement interfaces, supports DI, lazy init, testable. Static: can't inherit/mock.
When use Factory vs Builder?	Factory: choose which class to create. Builder: construct ONE complex object step by step.
Adapter vs Decorator vs Facade?	Adapter = convert interface. Decorator = add behavior. Facade = simplify subsystem.
Why Repository + Unit of Work?	Repo abstracts data access; UoW groups multiple repo operations in one transaction.
What is CQRS and when to use it?	Separate reads and writes. Use when read/write models differ significantly or for scale.
How does Strategy eliminate switch?	Replace a switch on type with IStrategy injection — add types by adding classes.
MediatR — what pattern does it implement?	Mediator pattern. Decouples senders (controllers) from handlers (business logic).
Thread-safe Singleton in C#?	Use Lazy<T> or static initializer — both guaranteed thread-safe in .NET.

★ Quick Cheat Sheet — Revision Page

⚡ **Purpose:** Pattern → Problem → Solution → When to Use. Memorize this table before your interview!

★ Design Patterns Quick Cheat Sheet		
Pattern	Problem Solved	Solution
Singleton	Need one shared instance	Private ctor + static instance
Factory	Create objects w/o new keyword	Factory method returns interface
Builder	Complex object construction	Chain of .Set() methods
Adapter	Incompatible interfaces	Wrapper class translates calls
Decorator	Add behavior dynamically	Wrap objects with same interface
Facade	Simplify complex subsystem	One simple entry-point class
Strategy	Switch algorithms at runtime	Inject IStrategy, call Execute()
Observer	Notify many on state change	Subscribe/Publish event model
Mediator	Too many direct couplings	Central hub routes messages

Pattern	Problem It Solves	Core Solution	When to Use	.NET Example
Singleton	Need single shared instance	Private ctor + static Lazy<T>	Logger, Config, Cache	AddSingleton()
Factory	Can't hardcode concrete class	Factory method returns interface	Payment types, connectors	IHttpClientFactory
Builder	Complex object with many params	Fluent .Set().Build() chain	API requests, Emails	WebApplicationBuilder
Adapter	Third-party incompatible API	Wrapper implementing your interface	External SDKs	Custom wrappers
Decorator	Add cross-cutting behavior	Wrap in class with same interface	Logging, Caching, Retry	Middleware pipeline
Facade	Complex multi-service calls	One orchestrating class	Checkout, Order flow	Service layer classes
Strategy	Many if/switch for behavior	IStrategy injection	Discounts, Sorting	IComparer<T>
Observer	Many listeners for one event	Subscribe/Notify list	Notifications, Events	C# events, MediatR
Mediator	Too many direct dependencies	Central hub routes all messages	CQRS, decoupled flows	MediatR library

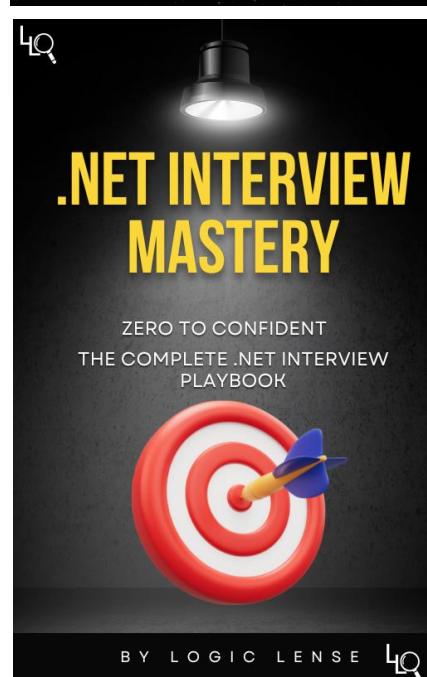
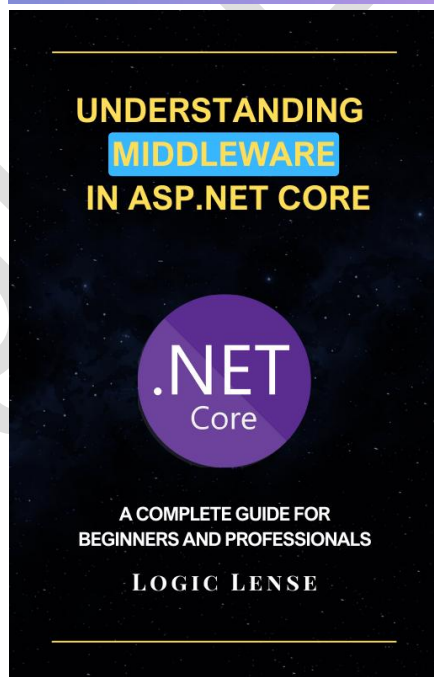
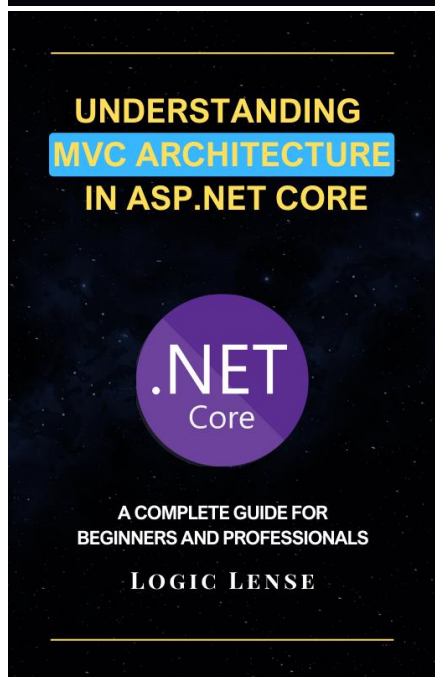
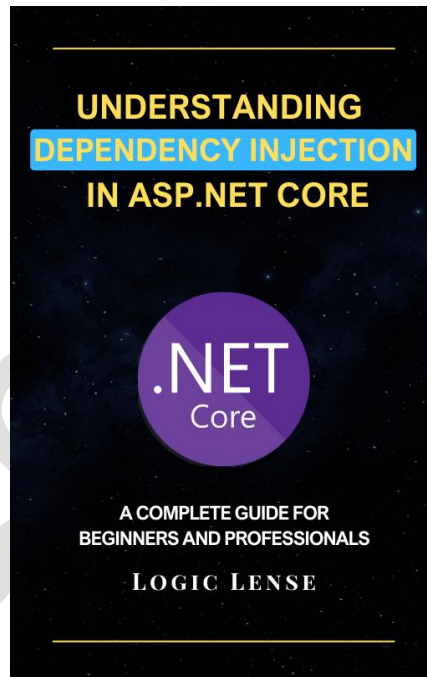
🔗 Interview Tip

For each pattern, always explain: PROBLEM → SOLUTION → REAL EXAMPLE → HOW
IN .NET

*"Singleton ensures one shared instance — like a Logger. In .NET, I use AddSingleton() or
Lazy<T>."*

Logic Lense

★ RECOMMENDED EBOOKS YOU WILL LOVE

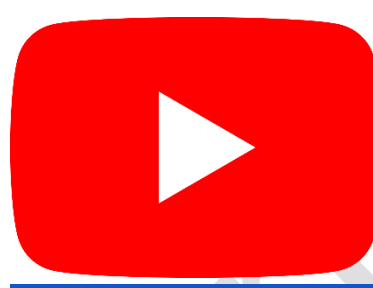


We Would love to have you as a learner ❤️🌟!

★ STAY CONNECTED & LEARN DAILY

Follow Logic Lense for daily content, tips, jobs, projects, and coding knowledge:

Share your feedback at: - mailto:support@logiclense.com



★ A FINAL NOTE

The tech world rewards **consistent learners**.
Stay committed. Stay curious. Keep building.
Your dream job is closer than you think.

See you in the next product.
Keep growing with Logic Lense. ❤️